

Bufio em Go: leitura e escrita eficiente com buffers

O pacote `bufio` da linguagem Go é utilizado para realizar operações de entrada e saída de dados de forma mais eficiente. Ele trabalha em conjunto com interfaces como `io.Reader` e `io.Writer`, adicionando uma área intermediária chamada **buffer**.

Na prática, em vez de o programa acessar um arquivo, teclado ou outro recurso a cada pequeno pedaço de informação, o `bufio` acumula dados temporariamente na memória. Isso pode melhorar bastante o desempenho, principalmente quando estamos trabalhando com muitos dados.

O pacote é importado desta forma:

```
import "bufio"
```

O que é um buffer?

Imagine que você precisa transportar 100 caixas de um lugar para outro.

Sem um buffer, seria como transportar **uma caixa por viagem**.

Com um buffer, seria como usar um caminhão e transportar **várias caixas de uma vez**.

No computador ocorre algo parecido. Fazer muitas operações pequenas de leitura ou escrita pode ser mais lento. O buffer reúne vários dados antes de realizar a operação.

O pacote `bufio` possui três estruturas muito utilizadas:

```
bufio.Reader  
bufio.Writer  
bufio.Scanner
```

Cada uma atende a uma necessidade diferente.

1. bufio.Reader

O `bufio.Reader` permite realizar leituras utilizando um buffer.

Ele pode ser utilizado para ler:

- arquivos;
- teclado;
- conexões de rede;
- qualquer estrutura que implemente `io.Reader`.

Um `Reader` pode ser criado com:

```
bufio.NewReader()
```

Exemplo: lendo dados digitados pelo usuário

```
package main  
  
import (  
    "bufio"
```

```

        "fmt"
        "os"
    )

    func main() {

        // Criamos um Reader que receberá dados do teclado
        leitor := bufio.NewReader(os.Stdin)

        fmt.Print("Digite seu nome: ")

        // Lê até encontrar uma quebra de linha
        nome, err := leitor.ReadString('\n')

        if err != nil {
            fmt.Println("Erro ao ler:", err)
            return
        }

        fmt.Println("Olá,", nome)
    }

```

A linha:

```
leitor := bufio.NewReader(os.Stdin)
```

cria um buffer para ler informações digitadas no terminal.

Já:

```
nome, err := leitor.ReadString('\n')
```

significa:

Leia caracteres até encontrar uma quebra de linha.

O caractere:

```
'\n'
```

representa a tecla **Enter**.

2. Limpando o Enter com strings.TrimSpace

Quando utilizamos `ReadString('\n')`, normalmente o Enter também faz parte da string recebida.

Podemos removê-lo usando:

```
strings.TrimSpace()
```

Exemplo:

```

package main

import (
    "bufio"
    "fmt"
    "os"
    "strings"
)

func main() {

```

```

    leitor := bufio.NewReader(os.Stdin)

    fmt.Print("Digite seu nome: ")

    nome, err := leitor.ReadString('\n')

    if err != nil {
        fmt.Println("Erro:", err)
        return
    }

    nome = strings.TrimSpace(nome)

    fmt.Println("Bem-vindo,", nome)
}

```

Agora a variável `nome` contém apenas o texto digitado.

3. Lendo um arquivo com `bufio.Reader`

Também podemos utilizar o `bufio.Reader` para ler arquivos.

Exemplo de arquivo chamado:

`dados.txt`

Conteúdo:

```

Go
Python
Java
C++

```

Programa:

```

package main

import (
    "bufio"
    "fmt"
    "io"
    "os"
)

func main() {

    arquivo, err := os.Open("dados.txt")

    if err != nil {
        fmt.Println("Erro ao abrir arquivo:", err)
        return
    }

    defer arquivo.Close()

    leitor := bufio.NewReader(arquivo)

    for {

        linha, err := leitor.ReadString('\n')

        fmt.Print(linha)
    }
}

```

```

        if err == io.EOF {
            break
        }

        if err != nil {
            fmt.Println("Erro durante leitura:", err)
            break
        }
    }
}

```

O comando:

```
bufio.NewReader(arquivo)
```

cria um Reader utilizando o arquivo como origem dos dados.

4. bufio.Scanner

Outra maneira muito comum de trabalhar com dados é utilizando:

```
bufio.Scanner
```

O Scanner é bastante conveniente quando queremos ler dados:

- linha por linha;
- palavra por palavra;
- token por token.

Um scanner é criado com:

```
bufio.NewScanner()
```

Exemplo: lendo um arquivo linha por linha

```

package main

import (
    "bufio"
    "fmt"
    "os"
)

func main() {

    arquivo, err := os.Open("dados.txt")

    if err != nil {
        fmt.Println("Erro:", err)
        return
    }

    defer arquivo.Close()

    scanner := bufio.NewScanner(arquivo)

    for scanner.Scan() {
        fmt.Println(scanner.Text())
    }
}

```

```

        if err := scanner.Err(); err != nil {
            fmt.Println("Erro durante a leitura:", err)
        }
    }
}

```

A linha:

```
scanner.Scan()
```

procura o próximo dado.

Por padrão, o `Scanner` trabalha **linha por linha**.

Já:

```
scanner.Text()
```

retorna o texto encontrado.

Por isso o trecho:

```

for scanner.Scan() {
    fmt.Println(scanner.Text())
}

```

pode ser entendido como:

Enquanto existir uma próxima linha, leia e mostre essa linha.

5. Lendo dados do teclado com Scanner

O `Scanner` também pode ser utilizado para ler informações digitadas pelo usuário.

```

package main

import (
    "bufio"
    "fmt"
    "os"
)

func main() {

    scanner := bufio.NewScanner(os.Stdin)

    fmt.Print("Digite seu nome: ")

    scanner.Scan()

    nome := scanner.Text()

    fmt.Println("Olá,", nome)
}

```

Esse código costuma ser simples para programas onde o usuário precisa digitar textos.

6. Scanner lendo palavras

Por padrão, o `Scanner` separa dados por linhas.

Podemos alterar esse comportamento.

Para separar por palavras utilizamos:

```
scanner.Split(bufio.ScanWords)
```

Exemplo:

```
package main

import (
    "bufio"
    "fmt"
    "os"
)

func main() {

    scanner := bufio.NewScanner(os.Stdin)

    scanner.Split(bufio.ScanWords)

    fmt.Println("Digite algumas palavras:")

    for scanner.Scan() {

        palavra := scanner.Text()

        fmt.Println("Palavra:", palavra)

    }

}
```

Se o usuário digitar:

```
Go é muito interessante
```

o programa exibirá:

```
Palavra: Go
Palavra: é
Palavra: muito
Palavra: interessante
```

7. bufio.Writer

O `bufio.Writer` é utilizado para realizar escritas utilizando um buffer.

Ele é criado com:

```
bufio.NewWriter()
```

Um ponto extremamente importante é que os dados podem permanecer temporariamente no buffer.

Por isso, normalmente devemos chamar:

```
Flush()
```

para garantir que os dados sejam realmente enviados ao destino.

Exemplo escrevendo em um arquivo

```
package main
```

```

import (
    "bufio"
    "fmt"
    "os"
)

func main() {

    arquivo, err := os.Create("dados.txt")

    if err != nil {
        fmt.Println("Erro:", err)
        return
    }

    defer arquivo.Close()

    escritor := bufio.NewWriter(arquivo)

    escritor.WriteString("Primeira linha\n")
    escritor.WriteString("Segunda linha\n")
    escritor.WriteString("Terceira linha\n")

    escritor.Flush()

    fmt.Println("Arquivo gravado com sucesso.")
}

```

O código:

```
escritor := bufio.NewWriter(arquivo)
```

cria um buffer associado ao arquivo.

Depois colocamos informações dentro dele:

```
escritor.WriteString("Primeira linha\n")
```

Finalmente:

```
escritor.Flush()
```

envia os dados que ainda estavam no buffer para o arquivo.

8. Por que Flush é importante?

Considere:

```
writer := bufio.NewWriter(arquivo)
```

```
writer.WriteString("Olá")
```

O texto pode ainda estar apenas na memória do buffer.

Quando chamamos:

```
writer.Flush()
```

estamos dizendo:

Envie tudo que está no buffer para o destino.

Uma prática comum é escrever:

```
defer writer.Flush()
```

Logo depois da criação do Writer.

Exemplo:

```
writer := bufio.NewWriter(arquivo)
defer writer.Flush()
```

Assim o `Flush()` será executado quando a função terminar.

9. Adicionando dados a um arquivo

Podemos combinar `bufio.Writer` com `os.OpenFile`.

```
package main

import (
    "bufio"
    "fmt"
    "os"
)

func main() {

    arquivo, err := os.OpenFile(
        "log.txt",
        os.O_APPEND|os.O_CREATE|os.O_WRONLY,
        0644,
    )

    if err != nil {
        fmt.Println("Erro:", err)
        return
    }

    defer arquivo.Close()

    writer := bufio.NewWriter(arquivo)
    defer writer.Flush()

    writer.WriteString("Programa executado\n")
}
```

As opções:

`os.O_APPEND`

adicionam dados ao final do arquivo.

`os.O_CREATE`

cria o arquivo caso ele não exista.

`os.O_WRONLY`

abre o arquivo somente para escrita.

10. Exemplo completo: cadastro simples

Podemos juntar `Reader` e `Writer` em um pequeno programa.

O programa perguntará o nome e a profissão do usuário e gravará os dados em um arquivo.

```
package main

import (
    "bufio"
    "fmt"
    "os"
    "strings"
)

func main() {

    // Reader para entrada pelo teclado
    leitor := bufio.NewReader(os.Stdin)

    fmt.Print("Nome: ")

    nome, err := leitor.ReadString('\n')

    if err != nil {
        fmt.Println("Erro:", err)
        return
    }

    nome = strings.TrimSpace(nome)

    fmt.Print("Profissão: ")

    profissao, err := leitor.ReadString('\n')

    if err != nil {
        fmt.Println("Erro:", err)
        return
    }

    profissao = strings.TrimSpace(profissao)

    // Abre ou cria o arquivo
    arquivo, err := os.OpenFile(
        "cadastro.txt",
        os.O_APPEND|os.O_CREATE|os.O_WRONLY,
        0644,
    )

    if err != nil {
        fmt.Println("Erro ao abrir arquivo:", err)
        return
    }

    defer arquivo.Close()

    // Writer com buffer
    writer := bufio.NewWriter(arquivo)
    defer writer.Flush()

    writer.WriteString(
        fmt.Sprintf(
            "Nome: %s | Profissão: %s\n",
            nome,
```

```

        profissao,
    ),
)

fmt.Println("Cadastro realizado com sucesso.")
}

```

Se o usuário informar:

Nome: Marcelo
Profissão: Programador

o arquivo poderá ficar assim:

Nome: Marcelo | Profissão: Programador

Executando novamente:

Nome: Maria
Profissão: Designer

o arquivo ficará:

Nome: Marcelo | Profissão: Programador
Nome: Maria | Profissão: Designer

Reader, Scanner ou Writer?

De maneira simples:

Estrutura	Utilização
<code>bufio.Reader</code>	Ler dados com maior controle
<code>bufio.Scanner</code>	Ler facilmente linha por linha ou palavra por palavra
<code>bufio.Writer</code>	Escrever dados utilizando um buffer

Para programas simples que precisam ler arquivos linha por linha, `Scanner` costuma ser bastante conveniente.

Quando precisamos controlar delimitadores ou fazer operações específicas de leitura, `Reader` oferece mais possibilidades.

Para escrita eficiente, podemos utilizar `Writer`.

Comparação entre Reader e Scanner

Considere que queremos ler:

João da Silva

Com `Reader`:

```

leitor := bufio.NewReader(os.Stdin)

nome, err := leitor.ReadString('\n')

```

Com `Scanner`:

```

scanner := bufio.NewScanner(os.Stdin)

scanner.Scan()

```

```
nome := scanner.Text()
```

O Scanner normalmente produz um código mais simples.

Já o Reader possui mais operações específicas, como:

```
ReadString()  
ReadBytes()  
ReadByte()  
ReadRune()  
Peek()
```

Exemplo de Peek

O método `Peek()` permite observar determinados bytes sem removê-los do buffer.

```
package main  
  
import (  
    "bufio"  
    "fmt"  
    "strings"  
)  
  
func main() {  
    leitor := bufio.NewReader(  
        strings.NewReader("Go é legal"),  
    )  
  
    dados, err := leitor.Peek(2)  
  
    if err != nil {  
        fmt.Println("Erro:", err)  
        return  
    }  
  
    fmt.Println(string(dados))  
}
```

Saída:

```
Go
```

Os caracteres continuam disponíveis para futuras leituras porque `Peek()` apenas observa os dados.

Exemplo simples de leitura e escrita juntas

Neste exemplo, vamos copiar o conteúdo de um arquivo para outro.

```
package main  
  
import (  
    "bufio"  
    "fmt"  
    "os"  
)  
  
func main() {
```

```

origem, err := os.Open("entrada.txt")

if err != nil {
    fmt.Println("Erro:", err)
    return
}

defer origem.Close()

destino, err := os.Create("saida.txt")

if err != nil {
    fmt.Println("Erro:", err)
    return
}

defer destino.Close()

scanner := bufio.NewScanner(origem)

writer := bufio.NewWriter(destino)
defer writer.Flush()

for scanner.Scan() {

    linha := scanner.Text()

    writer.WriteString(linha + "\n")
}

if err := scanner.Err(); err != nil {
    fmt.Println("Erro de leitura:", err)
    return
}

fmt.Println("Arquivo copiado com sucesso.")
}

```

Podemos visualizar o fluxo assim:

```

entrada.txt
  ↓
bufio.Scanner
  ↓
Programa Go
  ↓
bufio.Writer
  ↓
saida.txt

```

Cuidados importantes

Ao utilizar `bufio`, existem alguns pontos que merecem atenção.

No `Writer`, lembre-se do:

```
Flush()
```

Caso contrário, parte dos dados pode permanecer no buffer.

No `Scanner`, depois do `for`, é recomendável verificar:

```
scanner.Err()
```

Exemplo:

```
if err := scanner.Err(); err != nil {  
    fmt.Println("Erro:", err)  
}
```

Também é importante saber que o `Scanner` possui um limite padrão para o tamanho de cada token. Para arquivos com linhas muito grandes, podemos aumentar esse limite utilizando `Scanner.Buffer()`.

Exemplo:

```
scanner := bufio.NewScanner(arquivo)  
  
buffer := make([]byte, 1024)  
  
scanner.Buffer(buffer, 1024*1024)
```

Nesse exemplo, o scanner poderá trabalhar com tokens de até aproximadamente 1 MB.

Conclusão

O pacote `bufio` é uma ferramenta importante da biblioteca padrão do Go para trabalhar com entrada e saída de dados de forma eficiente.

Os três elementos mais importantes para um iniciante lembrar são:

```
bufio.NewReader()
```

para criar uma leitura com buffer;

```
bufio.NewScanner()
```

para ler facilmente linhas ou palavras;

e:

```
bufio.NewWriter()
```

para realizar escrita com buffer.

Uma maneira simples de memorizar é:

```
Reader → lê  
Scanner → procura e lê partes  
Writer → escreve
```

E no caso do `Writer`, nunca se esqueça:

```
writer.Flush()
```

porque o dado que você escreveu pode ainda estar esperando dentro do buffer antes de ser efetivamente gravado.